Hundred Days of Coding in C Programing

Day – 17

Name – Arayana Khare

Batch – 15

Sap id – 590042432

Question

Write a menu-driven C program using switch-case to perform digit-based operations on a positive integer.

Define one separate user-defined function for each menu option, with void return type and with a simple function call (without call by value/reference).

Each function should read the required input, process it, and display the result.

Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases.

1. **Generate the largest possible number using its digits.**
2. **Generate the smallest possible number using its digits.**
3. **Swap its first and last digits.**
4. **Check whether it is a palindrome number.**

Step – 1 Algorithm

1. Start.

2. Display the menu containing the four digit operations and Exit.

3. Read the user's choice.

4. Use switch-case to call the corresponding function.

5. For largest number: read the number, extract its digits, sort them in descending order, and display the result.

6. For smallest number: read the number, extract its digits, sort them in ascending order, place the smallest non-zero digit first when necessary, and display the result.

7. For swapping first and last digits: read the number, identify the first and last digits, swap them, and display the new number.

8. For palindrome: read the number, reverse it, compare it with the original, and display whether it is a palindrome.

9. Repeat the menu until Exit is selected.

10. Stop.

Step – 2 Test Case Table

Test Case	Menu Option	Input	Expected Output
TC01	1	53241	Largest = 54321
TC02	1	90871	Largest = 98710
TC03	2	53241	Smallest = 12345
TC04	2	90871	Smallest = 1789
TC05	3	12345	After swapping = 52341
TC06	3	9876	After swapping = 6879
TC07	4	1221	Palindrome
TC08	4	12345	Not a Palindrome

Step – 3 Write a code in C Programing

```
#include <stdio.h>

/* Function declarations */

void largestNumber();

void smallestNumber();

void swapFirstLast();

void checkPalindrome();

int main()

{

int choice;
```

```
do
{
printf("\n===== DIGIT BASED OPERATIONS =====\n");
printf("1. Generate Largest Possible Number\n");
printf("2. Generate Smallest Possible Number\n");
printf("3. Swap First and Last Digits\n");
printf("4. Check Palindrome\n");
printf("5. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);
switch (choice)
{
case 1:
largestNumber();
break;
case 2:
smallestNumber();
break;
case 3:
swapFirstLast();
break;
case 4:
checkPalindrome();
break;
case 5:
```

```
printf("Program terminated.\n");
break;
default:
printf("Invalid choice! Please try again.\n");
}
} while (choice != 5);
return 0;
}
/* Function to generate the largest possible number */
void largestNumber()
{
long long n;
int digits[20], count = 0;
int i, j, temp;
printf("Enter a positive integer: ");
scanf("%lld", &n);
while (n > 0)
{
digits[count] = n % 10;
n = n / 10;
count++;
}
for (i = 0; i < count - 1; i++)
{
for (j = i + 1; j < count; j++)
```

```
{
if (digits[i] < digits[j])
{
temp = digits[i];
digits[i] = digits[j];
digits[j] = temp;
}
}
}
printf("Largest possible number = ");
for (i = 0; i < count; i++)
printf("%d", digits[i]);
printf("\n");
}
/* Function to generate the smallest possible number */
void smallestNumber()
{
long long n;
int digits[20], count = 0;
int i, j, temp, firstNonZero;
printf("Enter a positive integer: ");
scanf("%lld", &n);
while (n > 0)
{
digits[count] = n % 10;
```

```
n = n / 10;
count++;
}
for (i = 0; i < count - 1; i++)
{
for (j = i + 1; j < count; j++)
{
if (digits[i] > digits[j])
{
temp = digits[i];
digits[i] = digits[j];
digits[j] = temp;
}
}
}
if (digits[0] == 0 && count > 1)
{
firstNonZero = 1;
while (firstNonZero < count && digits[firstNonZero] == 0)
firstNonZero++;
if (firstNonZero < count)
{
temp = digits[0];
digits[0] = digits[firstNonZero];
digits[firstNonZero] = temp;
```

```
}
}
printf("Smallest possible number = ");
for (i = 0; i < count; i++)
printf("%d", digits[i]);
printf("\n");
}
/* Function to swap first and last digits */
void swapFirstLast()
{
long long n, original, divisor = 1;
int first, last;
long long middle, result;
printf("Enter a positive integer: ");
scanf("%lld", &n);
original = n;
while (n / divisor >= 10)
divisor = divisor * 10;
first = original / divisor;
last = original % 10;
middle = (original % divisor) / 10;
result = last * divisor + middle * 10 + first;
printf("Number after swapping first and last digits = %lld\n", result);
}
/* Function to check palindrome */
```

```
void checkPalindrome()
{
long long n, original, reverse = 0, remainder;
printf("Enter a positive integer: ");
scanf("%lld", &n);
original = n;
while (n > 0)
{
remainder = n % 10;
reverse = reverse * 10 + remainder;
n = n / 10;
}
if (original == reverse)
printf("%lld is a Palindrome number.\n", original);
else
printf("%lld is not a Palindrome number.\n", original);
}
```

Step – 4 Compilation and Execution

Case: 1

```c
    printf("Enter a positive integer: ");

    scanf("%lld", &n);


    original = n;


    while (n > 0)

    {

        remainder = n % 10;

        reverse = reverse * 10 + remainder;

        n = n / 10;

    }


    if (original == reverse)

        printf("%lld is a Palindrome number.\n", original);

    else

        printf("%lld is not a Palindrome number.\n", origina
```

input

```
===== DIGIT BASED OPERATIONS =====
1. Generate Largest Possible Number
2. Generate Smallest Possible Number
3. Swap First and Last Digits
4. Check Palindrome
5. Exit
Enter your choice: 1
Enter a positive integer: 53241
Largest possible number = 54321
```

Case: 2

```c
printf("Enter a positive integer: ");

scanf("%lld", &n);


original = n;


while (n > 0)

{

    remainder = n % 10;

    reverse = reverse * 10 + remainder;

    n = n / 10;

}


if (original == reverse)

    printf("%lld is a Palindrome number.\n", original);

else

    printf("%lld is not a Palindrome number.\n", origina
```

input

```
===== DIGIT BASED OPERATIONS =====
1. Generate Largest Possible Number
2. Generate Smallest Possible Number
3. Swap First and Last Digits
4. Check Palindrome
5. Exit
Enter your choice: 1
Enter a positive integer: 98071
Largest possible number = 98710
```

Case: 3

```
    printf("Enter a positive integer: ");

    scanf("%lld", &n);


    original = n;


    while (n > 0)

    {

        remainder = n % 10;

        reverse = reverse * 10 + remainder;

        n = n / 10;

    }


    if (original == reverse)

        printf("%lld is a Palindrome number.\n", original);

    else

        printf("%lld is not a Palindrome number.\n", original
```

input

```
===== DIGIT BASED OPERATIONS =====
1. Generate Largest Possible Number
2. Generate Smallest Possible Number
3. Swap First and Last Digits
4. Check Palindrome
5. Exit
Enter your choice: 2
Enter a positive integer: 53241
Smallest possible number = 12345
```

Case: 4

```c
printf("Enter a positive integer: ");

scanf("%lld", &n);


original = n;


while (n > 0)

{

    remainder = n % 10;

    reverse = reverse * 10 + remainder;

    n = n / 10;

}


if (original == reverse)

    printf("%lld is a Palindrome number.\n", original);

else

    printf("%lld is not a Palindrome number.\n", original
```

input

```
===== DIGIT BASED OPERATIONS =====
1. Generate Largest Possible Number
2. Generate Smallest Possible Number
3. Swap First and Last Digits
4. Check Palindrome
5. Exit
Enter your choice: 2
Enter a positive integer: 90871
Smallest possible number = 10789
```

Case: 5

```c
    printf("Enter a positive integer: ");

    scanf("%lld", &n);


    original = n;


    while (n > 0)

    {

        remainder = n % 10;

        reverse = reverse * 10 + remainder;

        n = n / 10;

    }


    if (original == reverse)

        printf("%lld is a Palindrome number.\n", original);

    else

        printf("%lld is not a Palindrome number.\n", original)

```

input

```
===== DIGIT BASED OPERATIONS =====
1. Generate Largest Possible Number
2. Generate Smallest Possible Number
3. Swap First and Last Digits
4. Check Palindrome
5. Exit
Enter your choice: 3
Enter a positive integer: 12345
Number after swapping first and last digits = 52341
```

Case: 6

```c
    printf("Enter a positive integer: ");

    scanf("%lld", &n);


    original = n;


    while (n > 0)

    {

        remainder = n % 10;

        reverse = reverse * 10 + remainder;

        n = n / 10;

    }


    if (original == reverse)

        printf("%lld is a Palindrome number.\n", original);

    else

        printf("%lld is not a Palindrome number.\n", original)
```

input

```
===== DIGIT BASED OPERATIONS =====
1. Generate Largest Possible Number
2. Generate Smallest Possible Number
3. Swap First and Last Digits
4. Check Palindrome
5. Exit
Enter your choice: 3
Enter a positive integer: 9876
Number after swapping first and last digits = 6879
```

Case: 7

```c
    printf("Enter a positive integer: ");

    scanf("%lld", &n);


    original = n;


    while (n > 0)

    {

        remainder = n % 10;

        reverse = reverse * 10 + remainder;

        n = n / 10;

    }


    if (original == reverse)

        printf("%lld is a Palindrome number.\n", original);

    else

        printf("%lld is not a Palindrome number.\n", original)
```

input

```
===== DIGIT BASED OPERATIONS =====
1. Generate Largest Possible Number
2. Generate Smallest Possible Number
3. Swap First and Last Digits
4. Check Palindrome
5. Exit
Enter your choice: 4
Enter a positive integer: 1221
1221 is a Palindrome number.
```

Case: 8

```c
    printf("Enter a positive integer: ");

    scanf("%lld", &n);


    original = n;


    while (n > 0)

    {

        remainder = n % 10;

        reverse = reverse * 10 + remainder;

        n = n / 10;

    }


    if (original == reverse)

        printf("%lld is a Palindrome number.\n", original);

    else

        printf("%lld is not a Palindrome number.\n", origina
```

input

```
===== DIGIT BASED OPERATIONS =====
1. Generate Largest Possible Number
2. Generate Smallest Possible Number
3. Swap First and Last Digits
4. Check Palindrome
5. Exit
Enter your choice: 4
Enter a positive integer: 12345
12345 is not a Palindrome number.
```

Step – 5 Conclusion

The program successfully performs all four digit-based operations using a menu-driven switch-case structure. Each operation is implemented using a separate void user-defined function with a simple function call, satisfying the requirements of the HDOC Day-17 activity.